CRAUSER Antoine : TP145284
CESA Matthew-Frédérick : TP145283
BESNARD Clément : TP145279
GAUDISSARD Karl : TP145871

03/05/2026

Group : 10

ADVANCED WEB DEVELOPMENT PROJECT REPORT :

MCQoodle

Lecturer : Jeffery Jeselee Sijore

PARIS PANTHÉON-ASSAS UNIVERSITÉ

A·P·U
ASIA PACIFIC UNIVERSITY
OF TECHNOLOGY & INNOVATION

I) Introduction and the idea of the project

MCQoodle is a web-based learning platform developed as part of the Advanced Web Development project. The idea of the project is inspired by Moodle, but with a focus only on allowing teachers to manage multiple-choice quizzes and permitting students to practice or take exams online.

The platform is designed around three main types of users: students, teachers, and administrators. Each user type has a different role in the system. Students can access their enrolled courses, join new courses using a course code, complete MCQs, and view their results. Teachers are intended to create and manage courses, modules, and quizzes. Administrators can create user accounts and manage access to the platform.

a) The Objectives

The main objective of this project is to create a functional web application that supports online learning through courses and single or multiple-choice quizzes.

The specific objectives of the project are:

- To develop a role-based web application for students, teachers, and administrators
- To allow users to log in securely using their email and password
- To allow students to view the courses in which they are enrolled
- To allow students to join a course using a unique course code
- To allow students to complete MCQs in their courses
- To support both practice quizzes and exam quizzes
- To save students' attempts and select answers in the database
- To calculate and display quiz scores
- To allow administrators to create new users
- To organize the backend using routes, controllers, and models
- To store all application data in a relational MySQL database
-To have a well designed and responsive interface throughout the whole website

b) Scope

The scopes of the project are:

- User authentication and role-based redirection
- Student dashboard with enrolled courses
- Course enrollment using a course join code
- Course detail page showing available MCQs
- Quiz page with questions and answer options
- Support for single-choice and multiple-choice questions
- Score calculation after quiz submission
- Saving quiz attempts and selected answers
- Reopening a practice quiz and reviewing previous answers
- Exam quiz restriction, where the student can only submit once
- Admin dashboard with user creation or deletion
- Reusable header and footer components

c) End-Users Specifications

Our website should mainly be used by a university such as APU, so the 3 main end-users are going to be :

Students: Principal users, they can log in, enroll or view the courses they are enrolled in, and practice and take exams with the MCQs

Teachers: Secondary users, can configure courses and MCQs (questions, answers, points and time given)

Administrator: Not really an end-user, he manages the website users, can create them or delete them from the database.

Our project follows a client-server architecture with a local hosted database using Xampp.

We use Vue.js for the frontend, Express and Node for the backend, Vue Router for the routing and MySQL for the database.

Also, for the local hosting of the Database, we used XAMPP and to hash the passwords to ensure security we use bcryptjs.

We coded the whole project on VSCode, and as a collaboration platform we used GitHub to manage and modify the project simultaneously.

d) Major Functions

As a Moodle like website for MCQs, the majors functions we implemented are :
-User Authentication
Users can log in using their email and password. The backend checks the user's credentials against the database. Passwords are stored as hashed values for security. After a successful login, the user is redirected to their HomePage according to their role.

-Student, Teacher, Admin Dashboard
The student dashboard displays the list of courses in which the student is enrolled. Each course is displayed as a course card with basic information such as the title, description, course code, and publication status. Same for the teacher but he can create the courses, modify them and create MCQ's. Admin can add or delete users on his HomePage.

- Join Course Using Course Code
Students can join a course by entering a course code provided by a teacher. The backend verifies if the code exists in the Courses table. If the course exists and the student is not already enrolled, a new record is created in the CourseEnrollments table.

-Course Page

When a student opens a course, the course page displays the main course information and the list of MCQs available for that course. Each MCQ has a button allowing the student to open and complete it.

-Quiz / MCQ Functionality

The quiz page displays the questions and possible answers. Our system supports both single-choice and multiple-choice questions. After submission, the student's score is calculated based on the correct answers. For practice quizzes, students can review their answers and restart the quiz. For exam quizzes, students can submit only once and cannot review their answers after submission.

-Attempt and Answer Saving

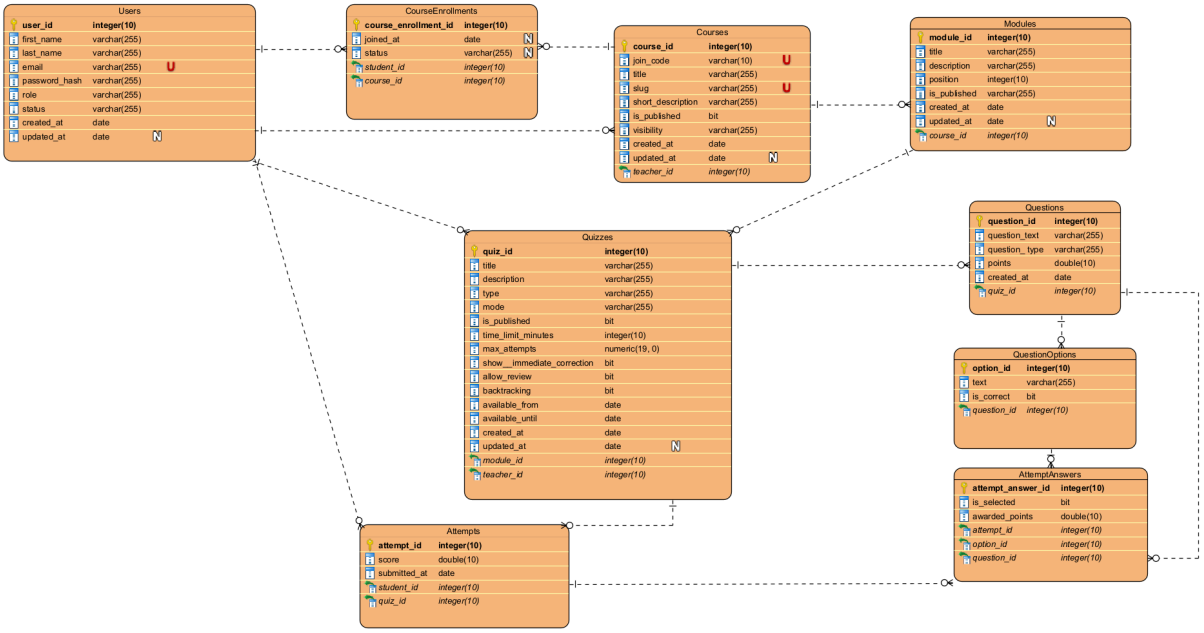
When a quiz is submitted, a new attempt is saved in the Attempts table. The selected answers are saved in the AttemptAnswers table. This allows the system to store the student's history and display previous results when needed.

-Reusable Layout Components

The web application uses reusable components such as a header and footer. We did this to improve consistency between pages and reduce duplicated code.

II) Design

a) ERD



This Entity Relationship Diagram represents the database structure of the MCQoodle platform. The database is organized around the main entities required for a learning and quiz system: users, courses, enrollments, modules, quizzes, questions, answer options, attempts, and attempt answers.

The Users table stores all platform accounts, including students, teachers, and administrators. A teacher can own several courses through the teacher_id foreign key in the Courses table. Students are linked to courses through the CourseEnrollments table, which allows the system to know which courses each student has joined.

Each course can contain several modules, and each module can contain several quizzes. A quiz is then linked to multiple questions, and each question has several

possible answer options stored in the QuestionOptions table. This structure allows the platform to support both single-choice and multiple-choice MCQs.

The Attempts table stores each quiz submission made by a student, including the score and submission date. The AttemptAnswers table stores the selected answer options for each attempt, making it possible to keep a record of the student's responses.

Some minor differences may appear in the SQL file from the project compared to the ERD, for example, SQL DATETIME fields may appear as date fields in the ERD, BOOLEAN values may appear as bit values, and ENUM fields may appear as varchar fields. In the actual SQL script, these fields are defined more precisely using DATETIME, BOOLEAN, ENUM, TEXT, and specific VARCHAR lengths.

b) Data Dictionary

Here is the composition of our database assembled in a data dictionary

1. TABLE: Users
Description: Stores user accounts, credentials, and roles.

- user_id (INT, PK): Unique identifier, auto-incremented.
- first_name (VARCHAR 100): User's first name.
- last_name (VARCHAR 100): User's last name.
- email (VARCHAR 255, UNIQUE): Unique email address used as login.
- password_hash (VARCHAR 255): Securely hashed password.
- role (ENUM): User role (admin, teacher, student). Default: student.
- status (ENUM): Account state (active, inactive, suspended).
- created_at (DATETIME): Timestamp when the account was created.
- updated_at (DATETIME): Timestamp of the last update (automatic).

2. TABLE: Courses
Description: List of courses created by teachers.

- course_id (INT, PK): Unique identifier for the course.
- join_code (VARCHAR 10, UNIQUE): Unique code used by students to enroll.
- title (VARCHAR 255): Name of the course.
- slug (VARCHAR 255, UNIQUE): URL-friendly version of the title.
- short_description (TEXT): Brief summary of the course content.
- is_published (BOOLEAN): Visibility status (0=Draft, 1=Published).
- visibility (ENUM): Access type (private, public, unlisted).
- teacher_id (INT, FK): Reference to Users(user_id) of the creator.

3. TABLE: CourseEnrollments

Description: Manages the relationship between students and courses.

- course_enrollment_id (INT, PK): Unique identifier for the enrollment.
- joined_at (DATETIME): Date and time the student joined.
- status (ENUM): Enrollment state (active, completed, dropped).
- student_id (INT, FK): Reference to Users(user_id).
- course_id (INT, FK): Reference to Courses(course_id).

4. TABLE: Modules

Description: Chapters or sections that organize a course.

- module_id (INT, PK): Unique identifier for the module.
- title (VARCHAR 255): Title of the module.
- description (TEXT): Detailed description of the module content.
- position (INT): Sorting order within the course.
- is_published (BOOLEAN): Whether the module is visible to students.
- course_id (INT, FK): Reference to Courses(course_id).

5. TABLE: Quizzes

Description: Assessments or quizzes linked to a module.

- quiz_id (INT, PK): Unique identifier for the quiz.
- title (VARCHAR 255): Name of the quiz.

- type (ENUM): Quiz nature (qcm, practice, exam).
- mode (ENUM): Question delivery (standard, randomized).
- is_published (BOOLEAN): Visibility status for students.
- time_limit_minutes (INT): Time allowed to complete the quiz.
- max_attempts (INT): Maximum number of attempts allowed per student.
- module_id (INT, FK): Reference to Modules(module_id).
- teacher_id (INT, FK): Reference to Users(user_id).

6. TABLE: Questions

Description: Individual questions belonging to a specific quiz.

- question_id (INT, PK): Unique identifier for the question.
- question_text (TEXT): The actual question wording.
- question_type (ENUM): Type (single_choice, multiple_choice).
- points (DECIMAL 10,2): Point value for the question (default 1.00).
- quiz_id (INT, FK): Reference to Quizzes(quiz_id).

7. TABLE: QuestionOptions

Description: Possible answers for each question.

- option_id (INT, PK): Unique identifier for the answer option.
- text (TEXT): The text of the answer option.
- is_correct (BOOLEAN): Flag indicating if this is a correct answer.
- question_id (INT, FK): Reference to Questions(question_id).

8. TABLE: Attempts

Description: Records of a student's session taking a quiz.

- attempt_id (INT, PK): Unique identifier for the attempt.
- score (DECIMAL 10,2): Total score achieved in this session.
- submitted_at (DATETIME): Date and time of submission.
- student_id (INT, FK): Reference to Users(user_id).
- quiz_id (INT, FK): Reference to Quizzes(quiz_id).

9. TABLE: AttemptAnswers

Description: Specific choices made by a student during an attempt.

- attempt_answer_id (INT, PK): Unique identifier for the record.
- is_selected (BOOLEAN): Whether the student selected this option.
- awarded_points (DECIMAL 10,2): Points earned for this specific answer.
- attempt_id (INT, FK): Reference to Attempts(attempt_id).
- option_id (INT, FK): Reference to QuestionOptions(option_id).
- question_id (INT, FK): Reference to Questions(question_id).

==========

DATABASE RELATIONSHIP RULES (ON DELETE)

==========

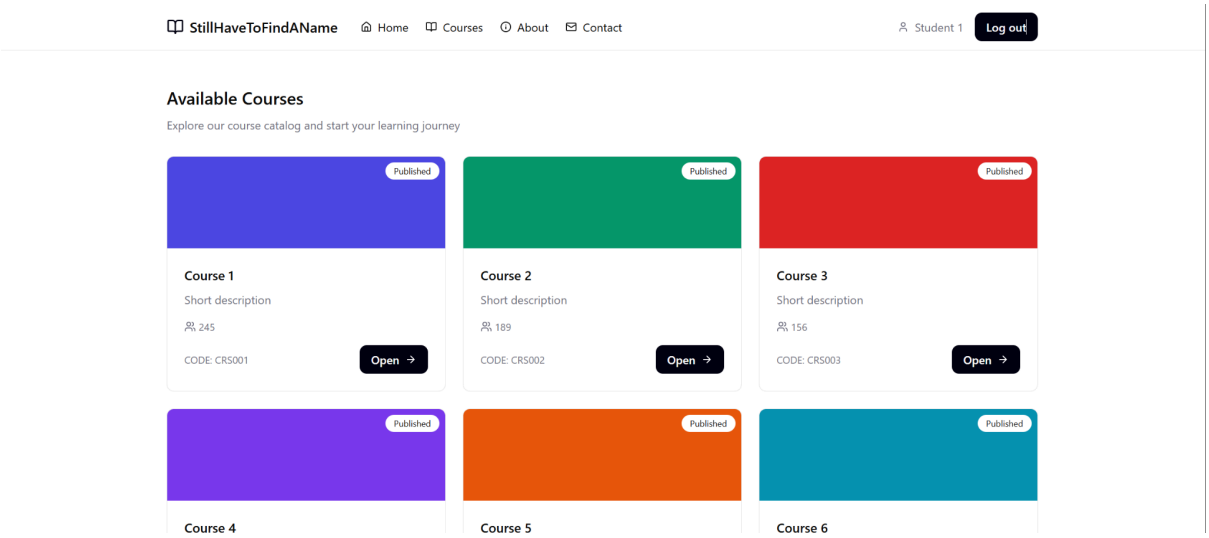
- Users -> Courses: SET NULL (Courses remain even if the teacher is deleted).
- Courses -> Modules/Enrollments: CASCADE (Deleting a course deletes its modules/enrollments).
- Modules -> Quizzes: SET NULL (Quizzes remain but are detached from the module).
- Quizzes -> Questions/Attempts: CASCADE (Deleting a quiz deletes all its questions/results).
- Questions -> Options/AttemptAnswers: CASCADE (Deleting a question deletes its options/history).

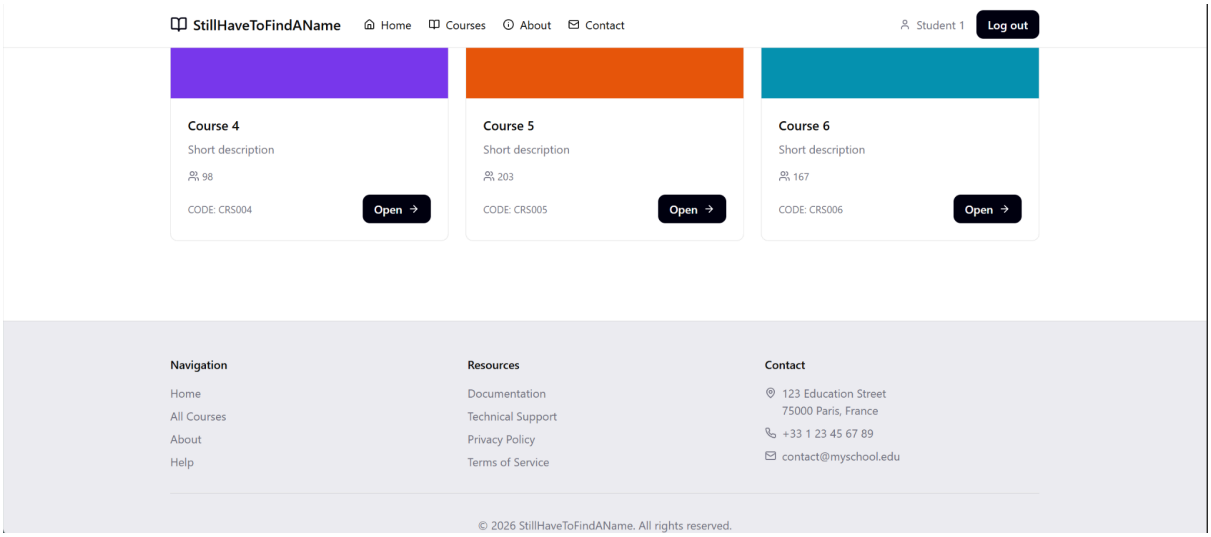
==========

c) Wireframe Layout

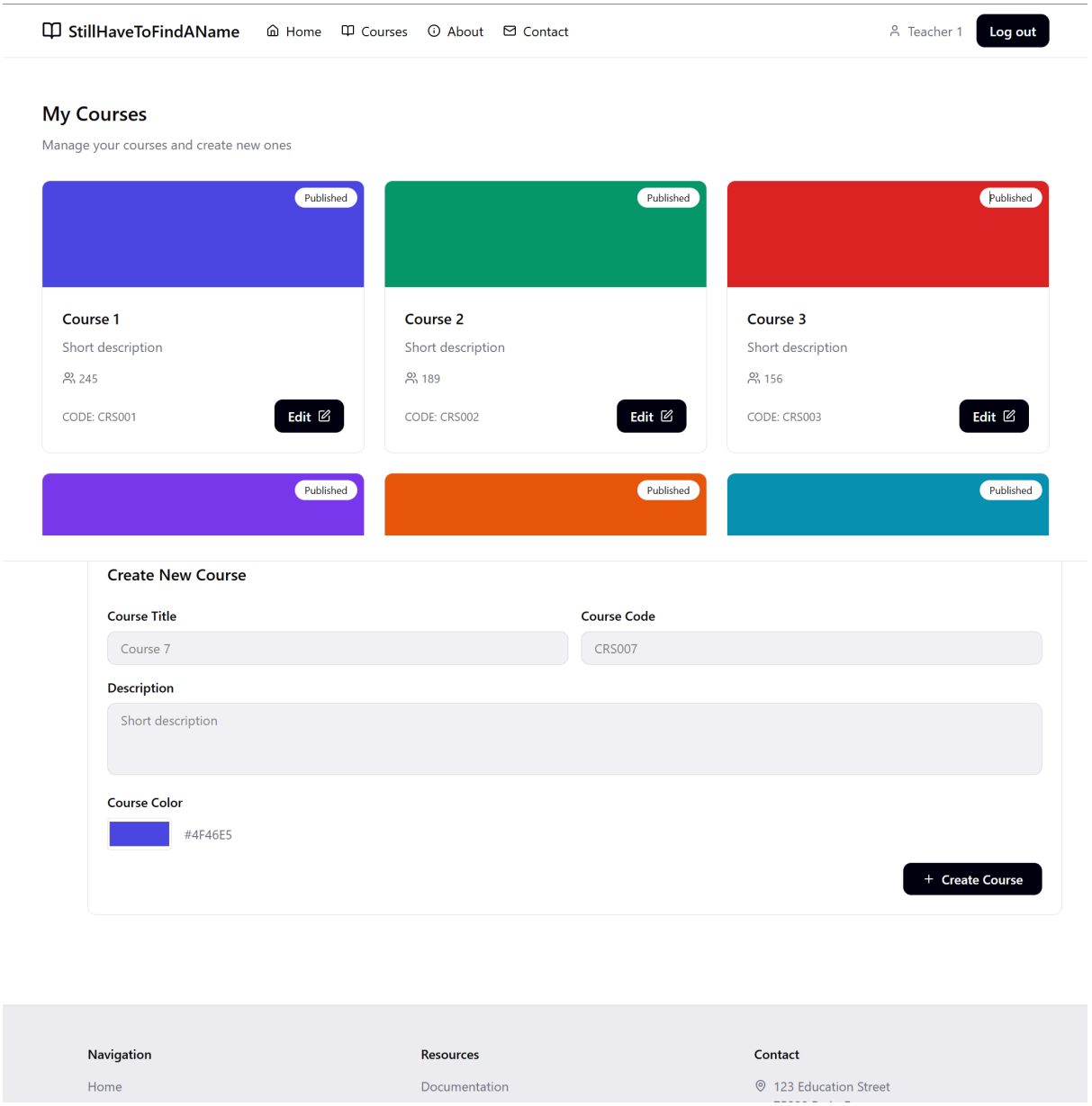
All of the following interfaces were designed on Figma.

Student :

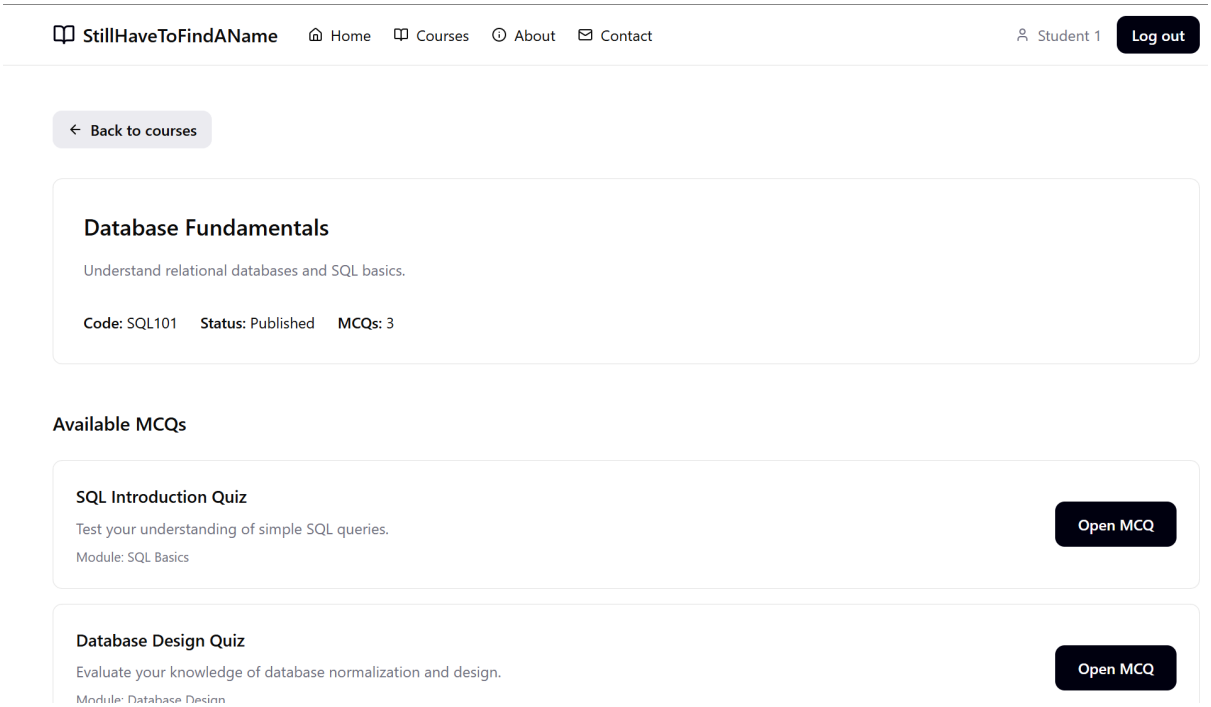




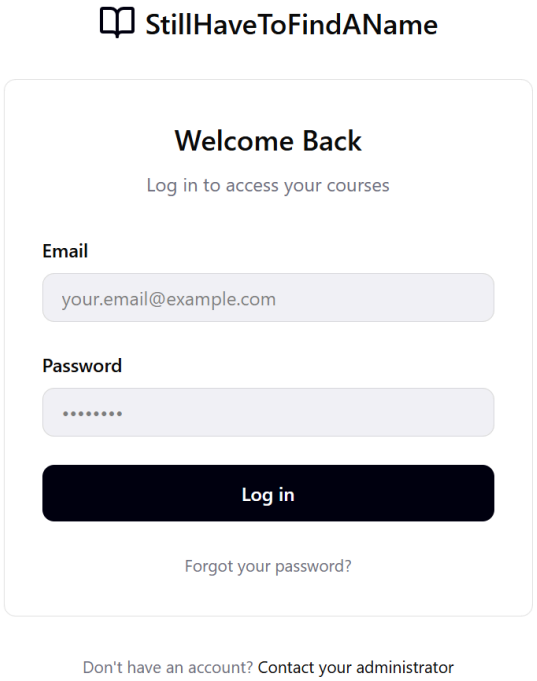
Teacher :



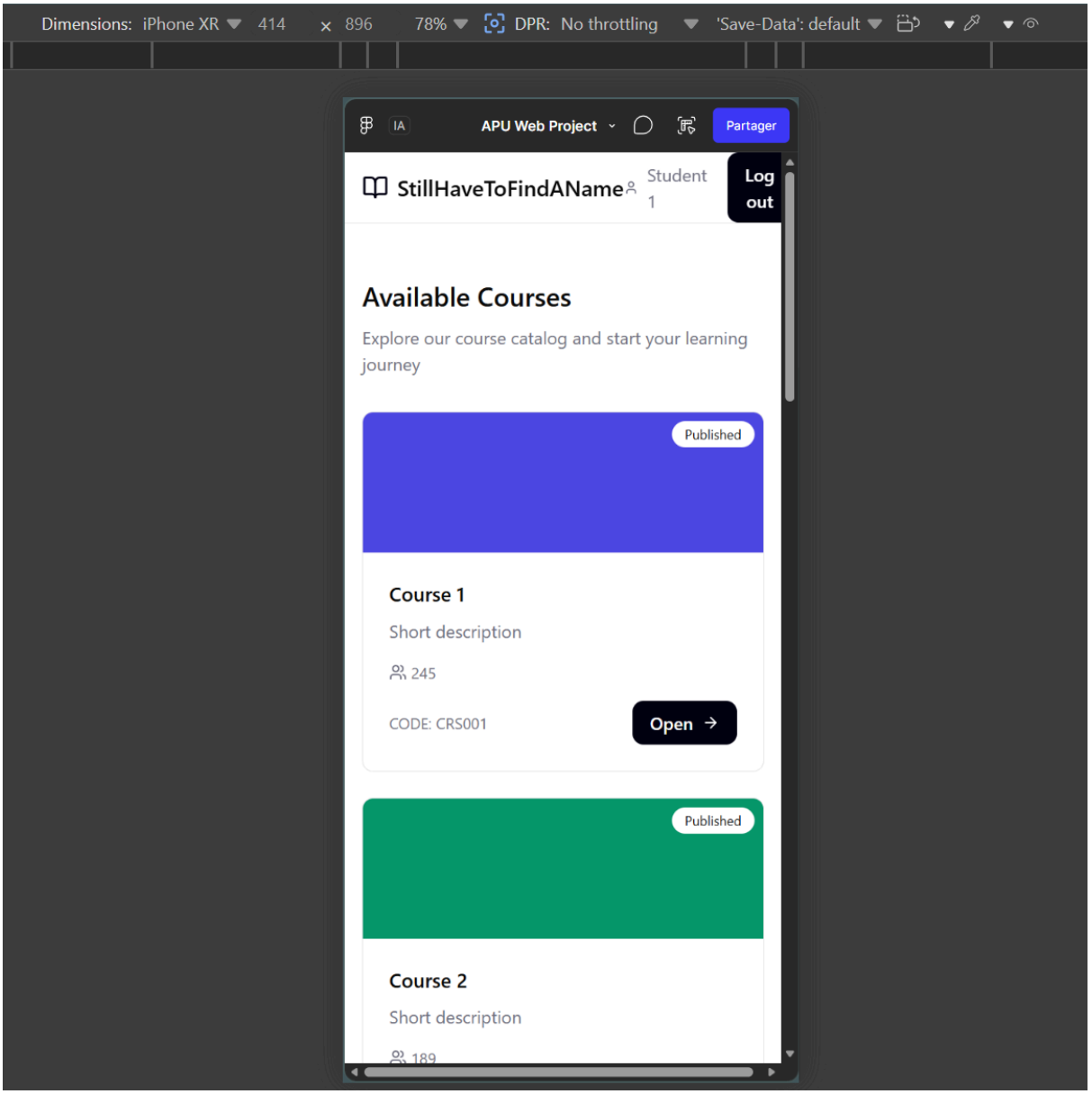
Open a Course from student perspective



and the Login page :



Also, it was designed to be responsive :



So we also made sure that the real website is fully responsive.

d) Website navigational structure

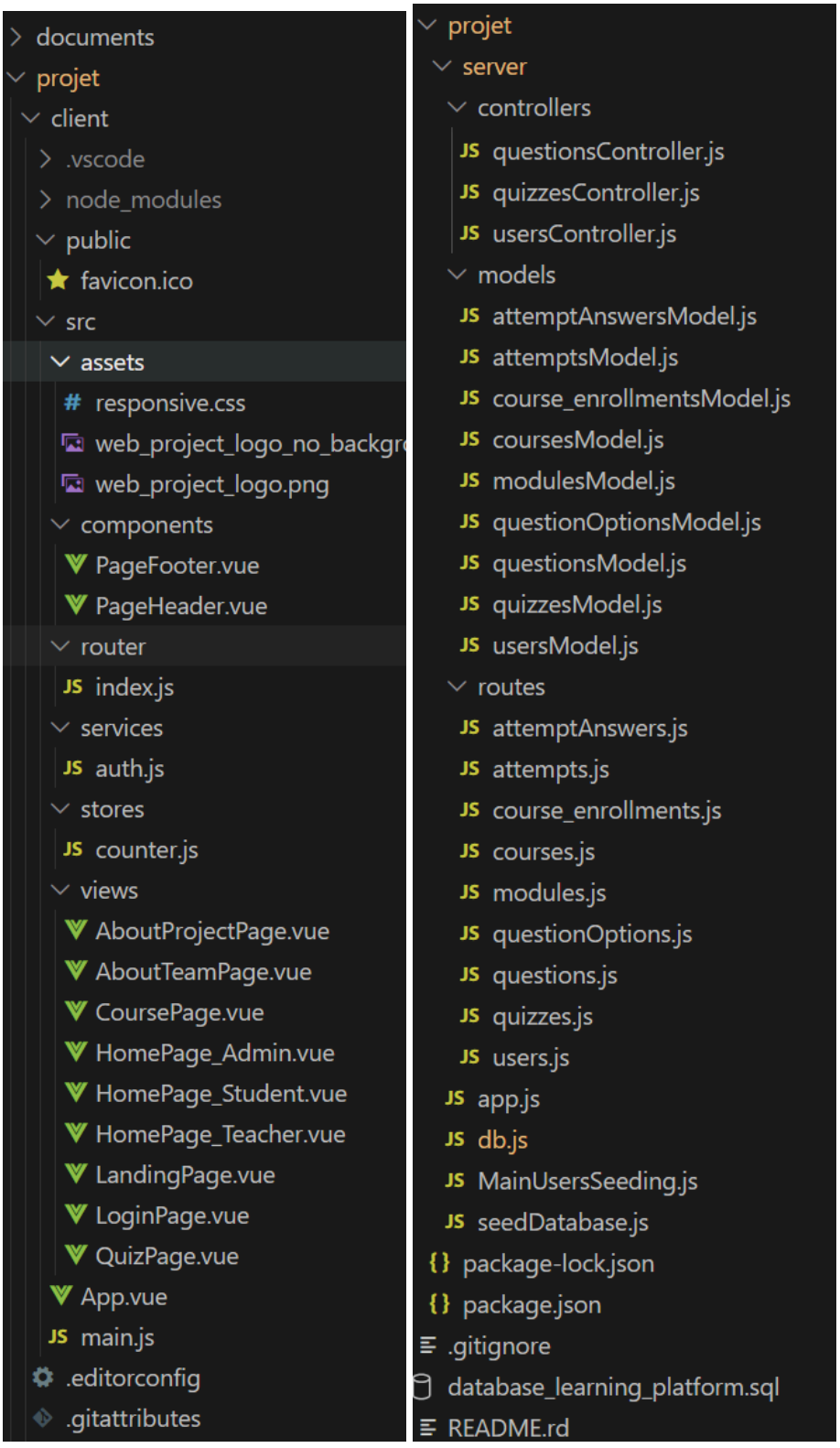
As a Navigation Diagram from the Landing page of our project, we have something like this :

-Landing page
- ->About our Project
- ->About our Team
- ->Login
  - ->>Student
    - ->>>Join Course with code
    - ->>>Course Page
      - ->>>>Quiz Page
    - ->>>LogOut
  - ->>Teacher
    - ->>>Create Course
    - ->>>Modify Course
      - ->>>>Create or Modify MCQ’s
    - ->>>LogOut
  - ->>Admin
    - ->>>Create or delete users
    - ->>>LogOut

When you click on the link in your terminal, you arrive on the Landing page from which you can see some information pages or Login buttons, after that, it depends with which user you logged in what you will have access to.

III) Implementation

a) Frontend and Backend Implementation



Our in MCQoodle web application The frontend and backend are separated into two main folders: client and server.

The client folder contains the Vue.js application. It includes the user interface, reusable components, views, and routing configuration. The server folder contains the Node.js and Express.js backend. It includes the API routes, controllers, models, database connection, and our seeding file.

-The views folder contains the main pages of the application, such as the landing page, login page, student/teacher/admin dashboard, course page, quiz page, and informational pages.
-The components folder contains reusable layout components that are : PageHeader.vue and PageFooter.vue.

The backend exposes several API endpoints, grouped by feature.
Each route group is handled by a specific route file inside the routes folder, and all the files are separated in 3 folders : the models, the controllers and all the routes are separated.

Also the database sql file and the db.js are there to initialise the database and use Xampp.

b) Authentification

The authentication feature is implemented using both frontend and backend logic. The frontend contains the login interface and sends the login request to the backend. The backend checks the submitted credentials, compares the password with the hashed password stored in the database, and returns the user information if the login is successful.
If the login fails, an error message is displayed on the page. If the login is successful, the returned user object is stored in localStorage. This allows the app to remember the logged-in user while navigating between pages.

Passwords are not stored directly in the database. Instead, we used bcryptjs to hash passwords before saving them. The Users table stores the hashed password in the password_hash column.

c) Enrollment Student Course

The course enrollment feature allows a student to view the courses they are already enrolled in and to join a new course using a course code.
The process can be summarized as:
Receive student_id and join_code
-> Validate input
-> Find course by join_code
-> Check if student is already enrolled
-> Create CourseEnrollments row
-> Return success response

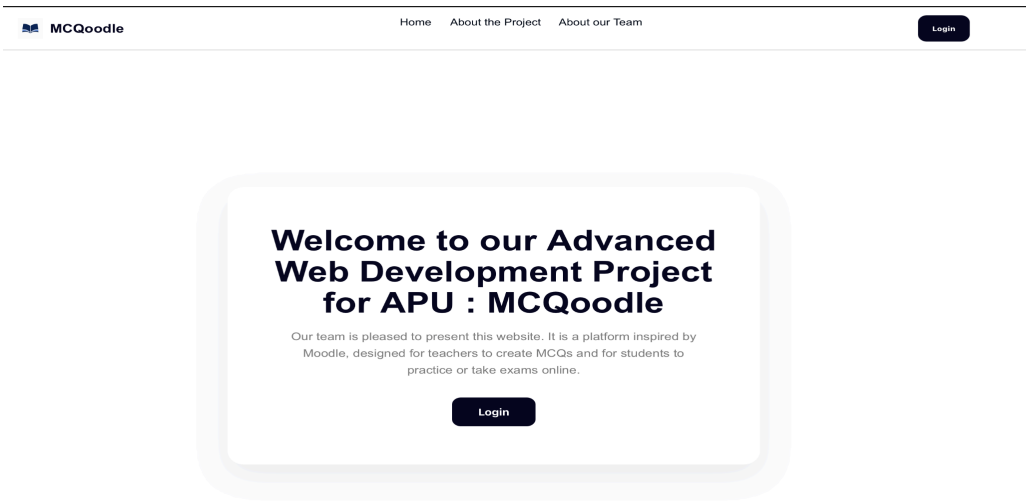
d) Courses and MCQ's

The Courses and MCQ feature allows students to move from a course to its quizzes, answer questions, submit their answers, and save their results. The implementation follows the following structure where courses contain modules, modules contain quizzes, quizzes contain questions, and questions contain options.

e) Admin creation and delete page

The admin dashboard allows him to manage platform users. He can create accounts with a fully integrated form for students, teachers, and other admins and can also delete accounts.

IV) User Guidance

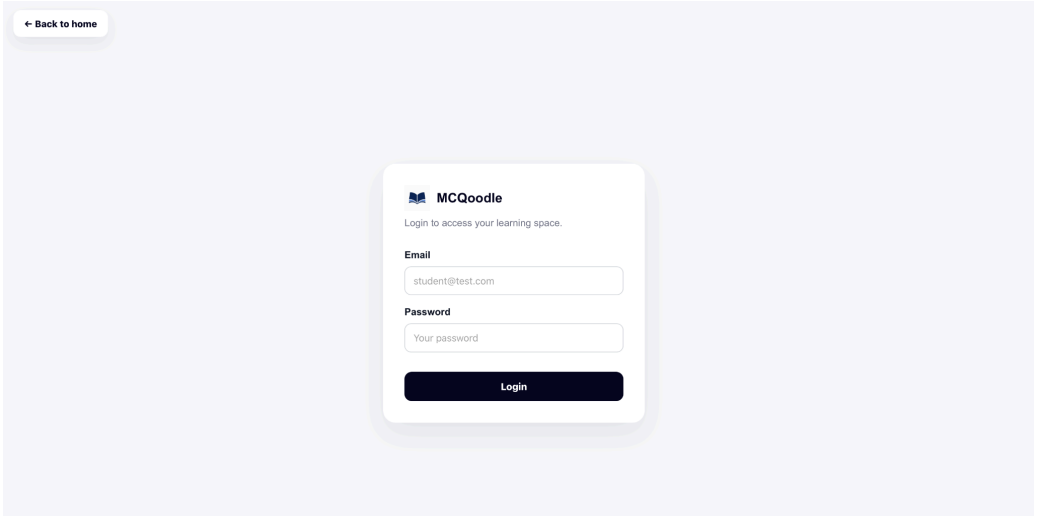


Landing Page

The landing page serves as the entry point of the MCQoodle platform. It presents the project to new visitors with a clear and minimalist design.

The navigation bar displays the platform name and logo, along with links to learn more about the project and the team, as well as a Login button for existing users.

The hero section introduces the platform with a title and a short description explaining that MCQoodle is a Moodle-inspired web application designed for teachers to create MCQ-based assessments and for students to practice or take exams online. A Login button invites users to access the platform directly from the homepage.

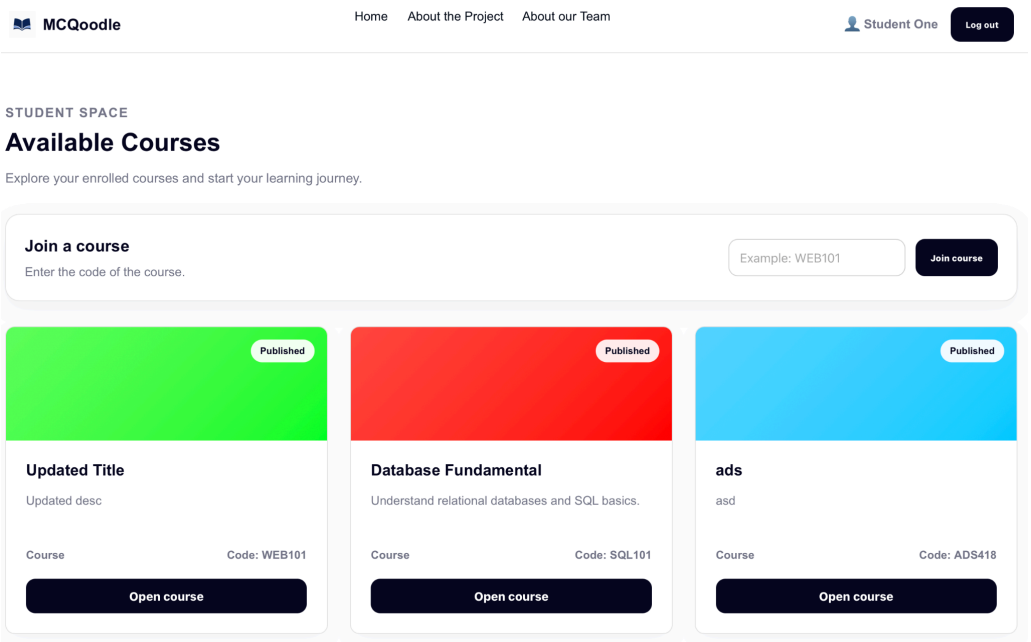


Login Page

The login page allows existing users to authenticate and access their personal learning space.

The page features a centered card with the MCQoodle logo and a short tagline inviting the user to log in. A "Back to home" button in the top left allows users to return to the landing page at any time.

The login form contains two fields: an email field and a password field. Once filled in, the user clicks the Login button to submit their credentials. Based on their role, student, teacher, or admin, they are then redirected to their respective dashboard.

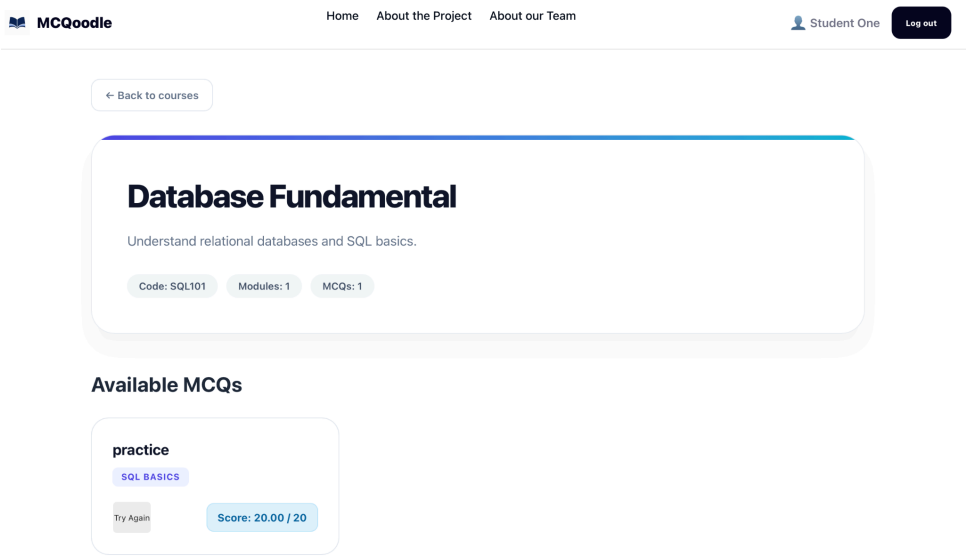


Student Dashboard

Once logged in, the student is redirected to their personal dashboard. The header displays their name and a Logout button to end the session.

The dashboard is divided into two main sections. The first is a "Join a course" panel, where students can enroll in a new course by entering a course code (example : WEB101) and clicking the Join course button. The second section displays all the courses the student is currently enrolled in as a grid of cards. Each card shows the course title, a short description, its join code, and a colored cover banner indicating

whether the course is published or not. Clicking on a card or the "Open course" button navigates the student to the course detail page.

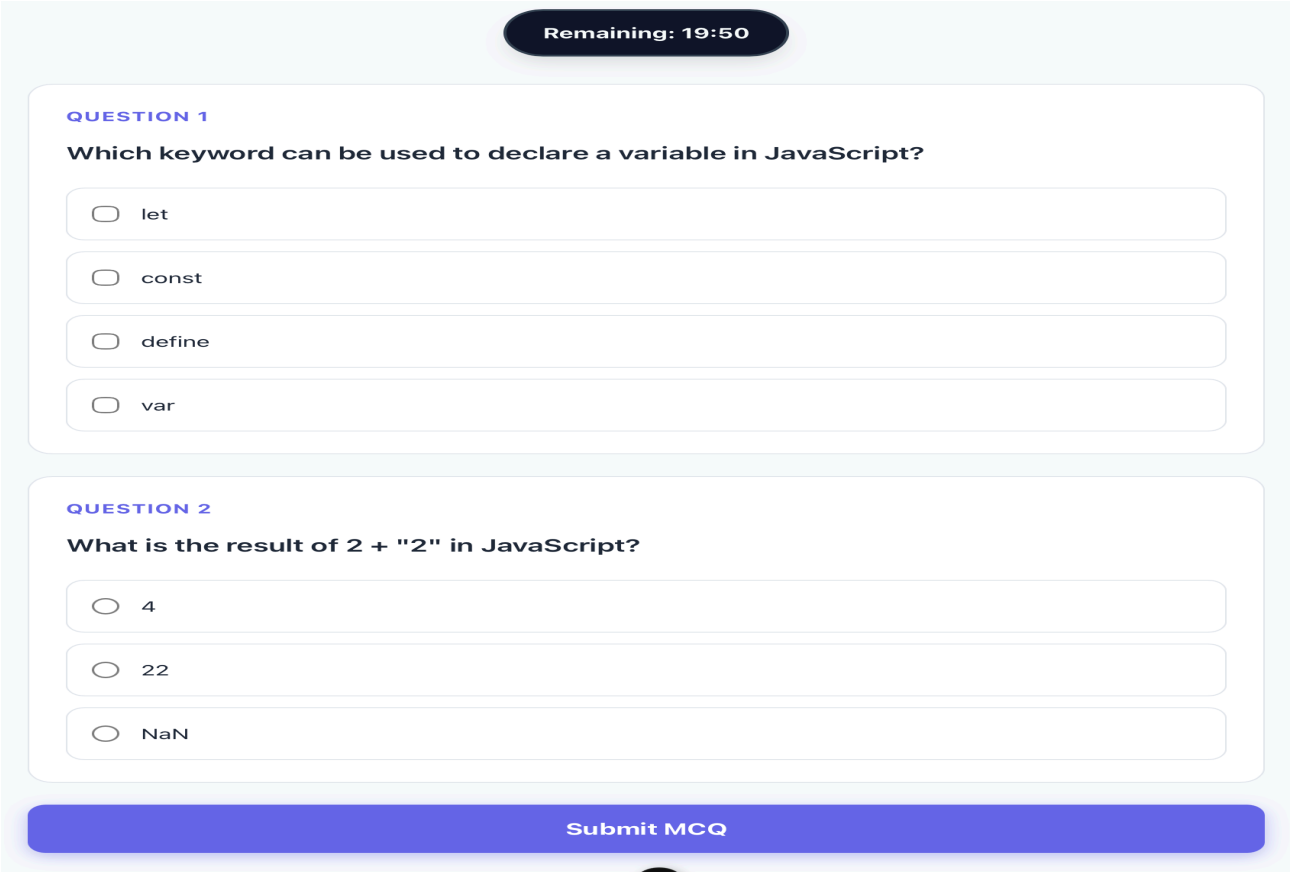


Course Detail Page - Student View

When a student opens a course, they are taken to the course detail page. A "Back to courses" button allows them to return to their dashboard at any time.

The top section displays the course header with its title, short description, and three key stats: the join code, the number of modules, and the number of MCQs available.

Below, the "Available MCQs" section lists all the quizzes attached to the course. Each MCQ card shows the quiz title, its module tag, and the student's last score. There are two types of MCQs on the platform: practice quizzes, which the student can attempt as many times as they want, and exam quizzes, which can only be taken once. In both cases, the student's most recent score is displayed directly on the card. If the quiz has already been attempted, a "Try Again" button appears for practice quizzes, allowing the student to retake it.



MCQ Page - Student View

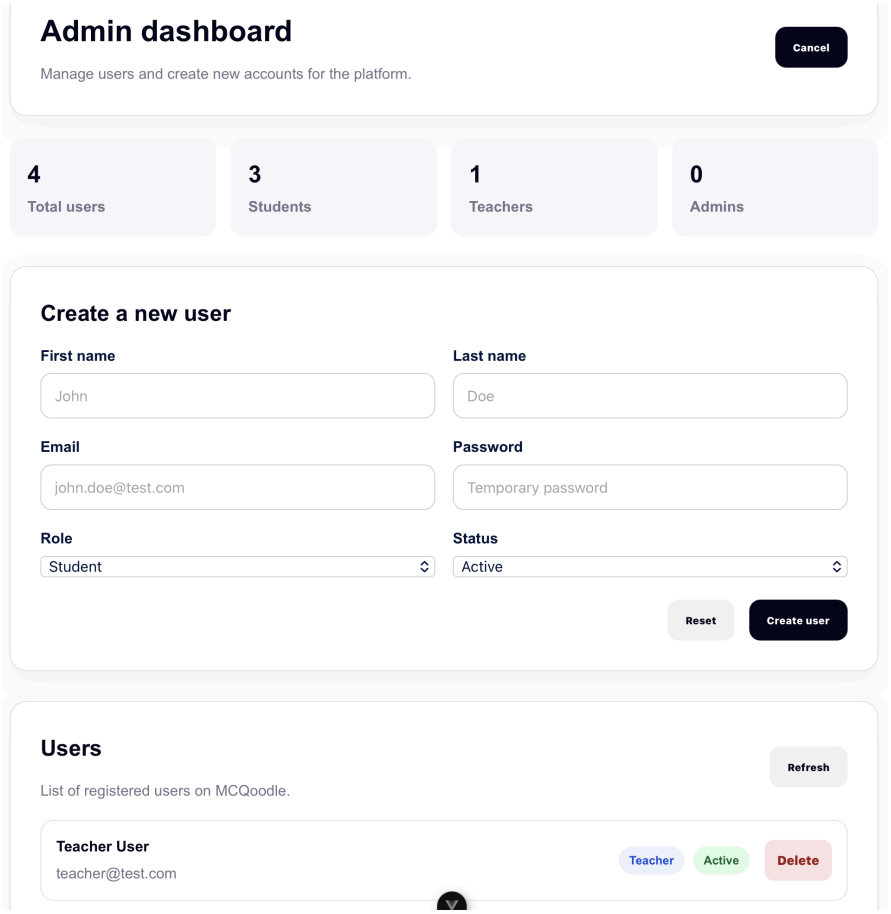
When a student starts a quiz, they are taken to the MCQ page where all questions are displayed one after another on a single scrollable page.

Each question shows its number, title, and a list of answer options. The platform supports two types of questions: single choice, where the student selects only one answer using a radio button, and multiple choice, where several answers can be selected. Each question carries a defined number of points that contribute to the final score.

A timer is visible at the top of the page and counts down for the duration of the quiz. If the timer runs out before the student finishes, all unanswered questions are automatically scored as zero and the quiz is submitted for evaluation without any further action required from the student.

Once the student has answered all questions, they click the "Submit MCQ" button at the bottom of the page to submit their attempt. After submission, the final score is displayed on the page. If the quiz is of type practice, a "Try Again" button appears, allowing the student to retake it as many times as they wish. If the quiz is of type

exam, the student is asked to leave the page and will no longer be able to access or retake that quiz.



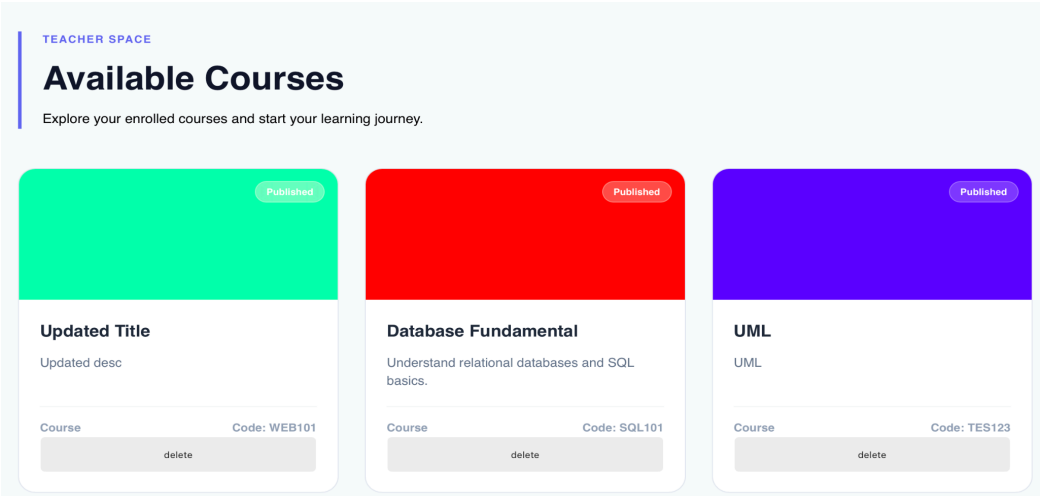
Admin Dashboard

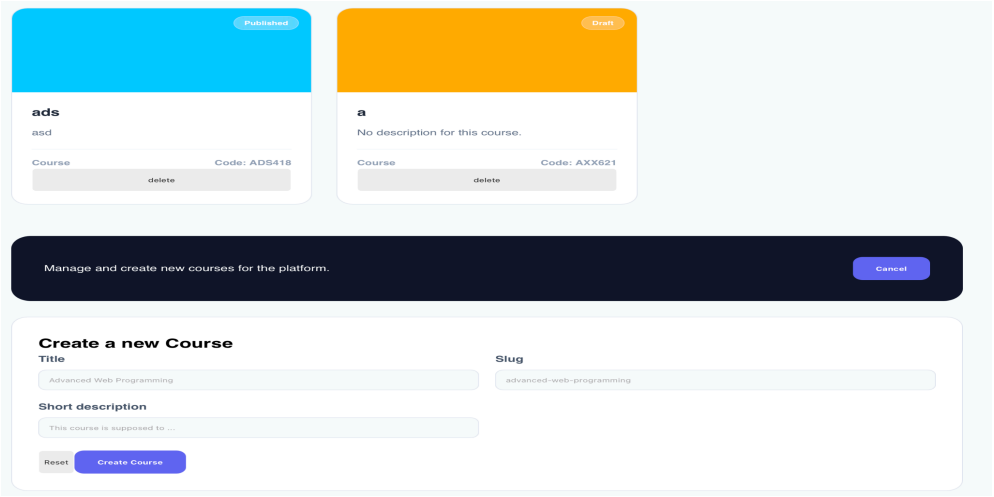
Let’s log out and now go to the login page to connect, this time as and admin. After logging in as an admin, the user is redirected to the admin dashboard, which is dedicated to user management across the platform.

At the top, four stat cards provide a quick overview of the platform's user base, displaying the total number of users, broken down by students, teachers, and admins.

Below, a "Create a new user" form allows the admin to register new accounts directly from the dashboard. The form collects the user's first name, last name, email address, and a temporary password. The admin also assigns a role student, teacher, or admin and sets the account status to either active or inactive. A Reset button clears the form and a Create user button submits it.

At the bottom, a Users section lists all registered accounts on the platform. Each entry displays the user's full name, email address, their role as a badge, and their current status. A Delete button allows the admin to permanently remove a user from the platform. A Refresh button reloads the list to reflect any recent changes.





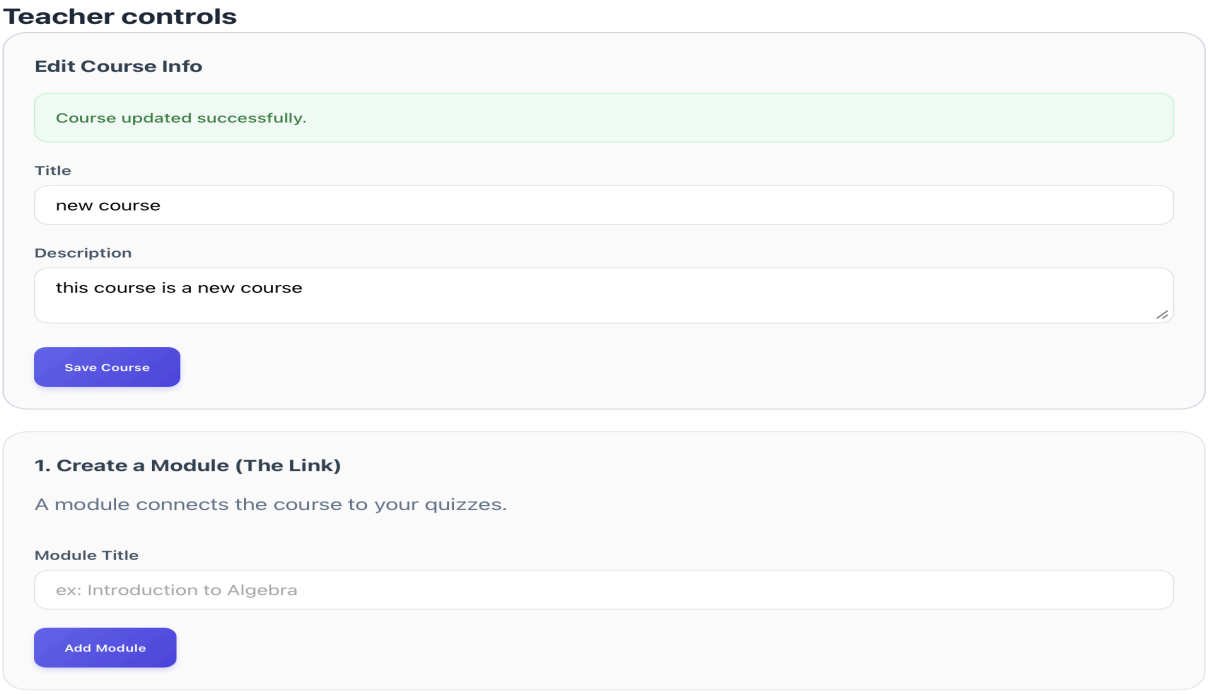
Teacher dashboard

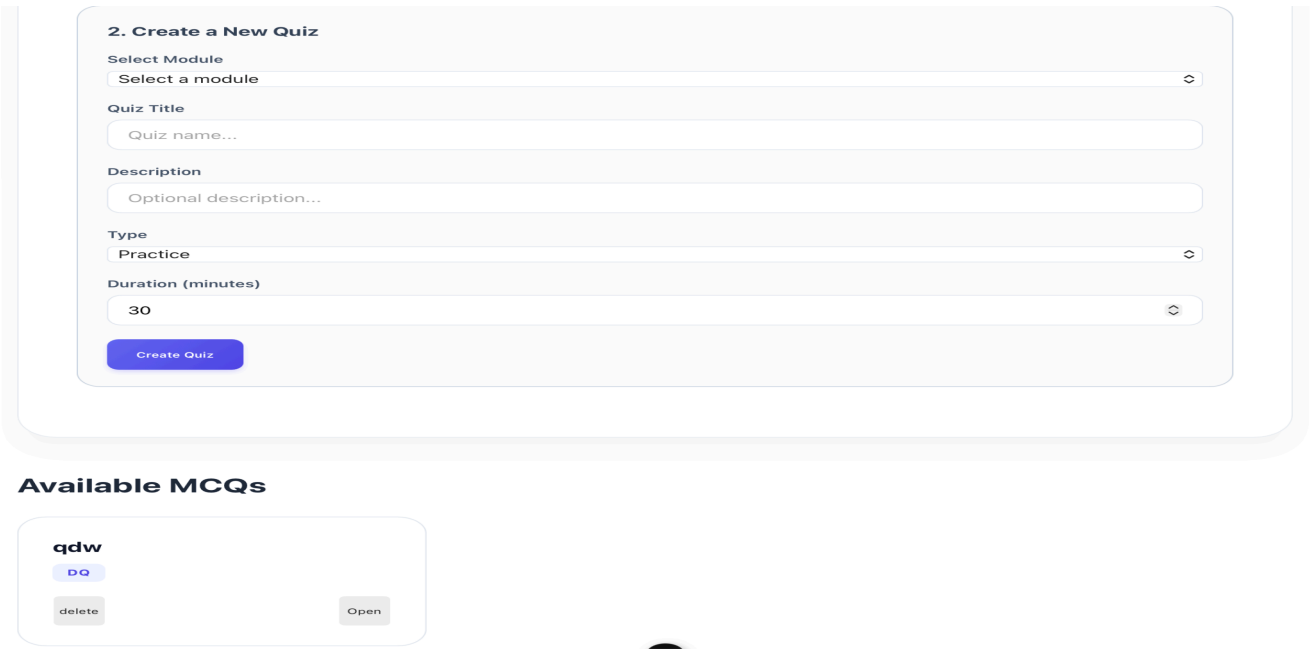
We can log out again.

After logging in as a teacher, the user is redirected to their personal dashboard. The Teacher Space displays all the courses they have created in a grid of cards.

Each card shows the course title, a short description, its join code, and a colored cover banner indicating whether the course is published or not. Unlike the student view, each card features a Delete button, allowing the teacher to permanently remove a course from the platform. Clicking on a card navigates the teacher to the course management page where they can edit and manage its content.

At the bottom of the page, a "Create a new course" form allows the teacher to add a new course directly from their dashboard. The form requires a course title, a slug, and a short description. Upon clicking the Create course button, the new course is immediately created and appears in the course grid without needing to reload the page.

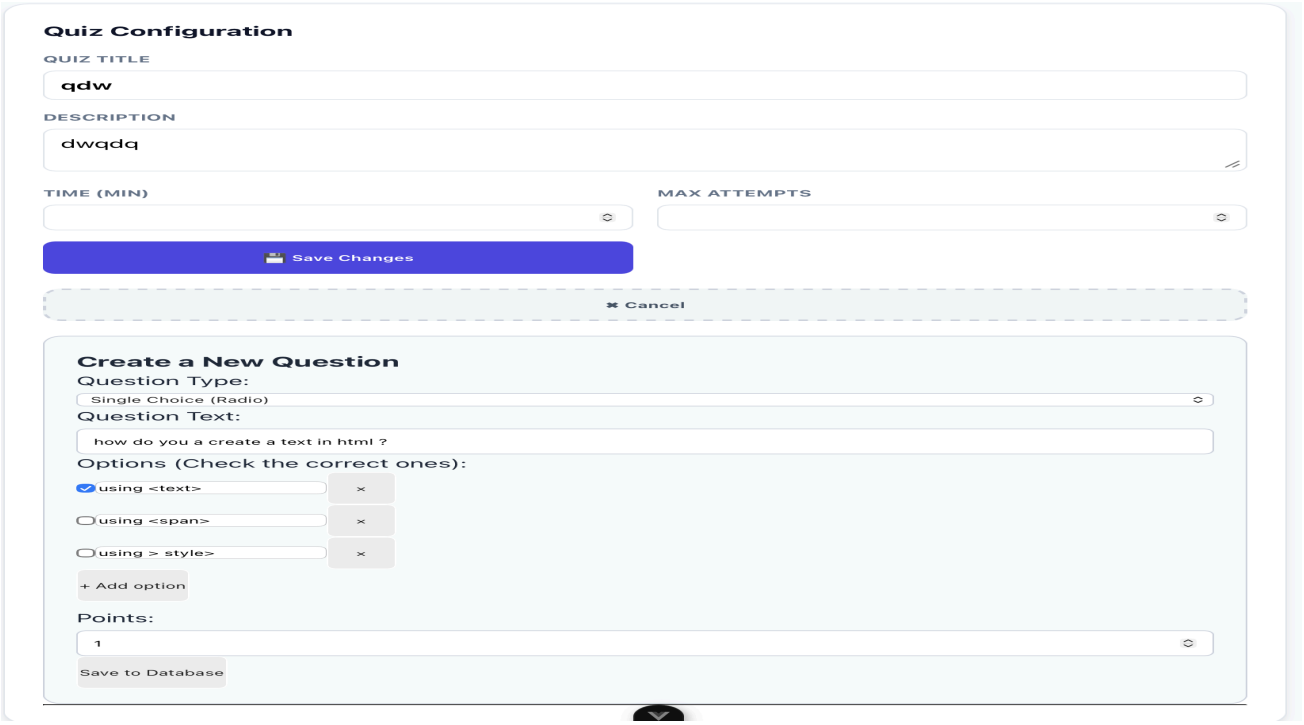


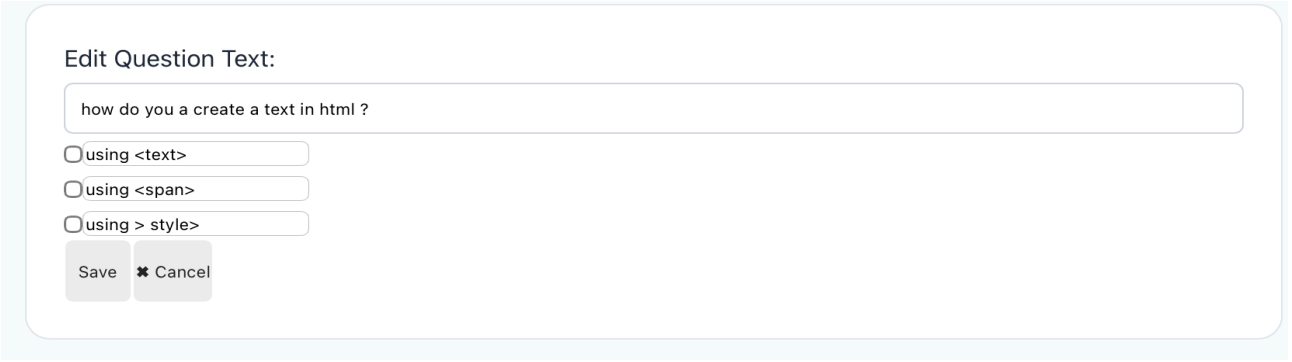


Course Management Page - Teacher View

When a teacher clicks on a course, they are taken to the course management page, also called the Teacher Controls panel. This page is structured as a step-by-step workflow for building a complete course. The first section, "Edit Course Info", allows the teacher to update the course title and description at any time. A success message is displayed upon saving to confirm the changes were applied. The second section, "Create a Module", allows the teacher to add modules to the course. A module acts as a category or chapter that groups quizzes together. The teacher simply enters a module title and clicks Add Module.The third section, "Create a New Quiz", allows the teacher to create an MCQ quiz linked to one of the course's modules. The form requires selecting a module, entering a quiz title, an optional description, choosing the type, either Practice or Exam, and setting a duration in minutes. Clicking Create Quiz adds the quiz to the course.At the bottom, the "Available MCQs" section lists all existing quizzes for that course. Each quiz card displays its title, the module it belongs to, and two action buttons: Delete to remove the quiz, and Open to access the quiz editor where the teacher can add and manage questions

.





Quiz Editor Page - Teacher ViewWhen a teacher clicks "Open" on a quiz card, they are taken to the Quiz Editor page, which is divided into two main sections.

The first section, "Quiz Configuration", allows the teacher to edit the quiz settings at any time. They can update the title, description, time limit in minutes, and the maximum number of attempts. The number of attempts directly determines the quiz type: if set to 1, the quiz is treated as an Exam and students can only take it once. If set to more than 1, it becomes a Practice quiz and students can retake it multiple times. Clicking Save Changes applies the updates, and a Cancel button discards them.

The second section, "Create a New Question", allows the teacher to add questions to the quiz. The teacher selects the question type, either Single Choice (radio buttons) or Multiple Choice (checkboxes), then enters the question text. They can

add as many answer options as needed using the "+ Add option" button, and mark the correct answer by checking the corresponding box next to each option. Each question is also assigned a point value. Clicking "Save to Database" saves the question.

Existing questions are listed below the creation form and can be edited directly. Clicking on an existing question opens an inline edit form where the teacher can update the question text and answer options, then save or cancel the changes.

V) Conclusion

MCQoodle successfully provides the core features we wanted it to have for it to be a learning platform. Including role-based login, course enrollment, course display, MCQ creation and completion, score calculation, some error handling, and user creation by administrators, we all worked together to achieve a result that satisfies all of us.
We were able to use Vue.js for the frontend, Express.js for the backend, and MySQL for relational data storage that we locally hosted with Xampp.

For the future enhancement we would have done if we had more time, we would have taken care of :
- Password reset functionality.
- Student progress tracking.
- Course materials and file uploads.
-And modifying user informations with the function we already had created but that never worked

VI) References

Vue.js : https://vuejs.org/ Express.js. : https://expressjs.com/

MySQL. : https://dev.mysql.com/doc/ Node.js. : https://nodejs.org/
Figma : https://www.figma.com/ for the wireframes and project model
ChatGPT for debugging, some CSS and the data in the seeder for the database.